

PERSONAL EXPENSE & BUDGET TRACKER

Subject: Programming in Java

Student Name	Harish Kumar
Registration Number	25BAI11056
School	SCAI

Academic Project Report

1. INTRODUCTION

Personal financial management is an important part of everyday life. Individuals make expenditures on food, transportation, education, entertainment, shopping, healthcare, bills, and other requirements. When these transactions are not recorded systematically, it becomes difficult to determine where money is being spent and whether expenditure remains within a planned budget.

Traditional methods such as handwritten notebooks and manually maintained spreadsheets can become inconvenient as the number of transactions increases. Manual tracking may result in forgotten transactions, arithmetic errors, duplicate entries, or difficulty in finding older records. It also requires additional effort to calculate category-wise totals and determine remaining budget.

The Personal Expense & Budget Tracker addresses these problems through a lightweight command-line software solution developed using Core Java. The application enables users to add, view, update, and delete expense records, set a spending budget, monitor expenditure, receive budget warnings, and generate summaries.

The application uses CSV-based File I/O for persistence. Expense information is stored in a file so that records remain available after the program is closed. The modular architecture separates data models, business logic, validation, storage, and user interaction.

From an academic perspective, the project demonstrates classes and objects, encapsulation, enums, collections, file operations, exception handling, input validation, and software testing.

2. PROBLEM STATEMENT

Managing daily expenses manually is time-consuming and error-prone. Users may forget transactions, make calculation mistakes, lose track of category-wise expenditure, or discover too late that they have exceeded their intended spending limit.

- Difficulty maintaining records: transactions may be forgotten or recorded inconsistently.
- Calculation errors: manually adding many transactions can produce incorrect totals.
- Lack of budget monitoring: users may not know how much of a planned budget has already been consumed.
- Limited spending analysis: manual records make category-wise analysis inconvenient.
- Modification difficulty: correcting or deleting old transactions can be cumbersome.
- No immediate alerts: conventional records do not automatically warn users when spending approaches or exceeds a limit.
- Poor organization: large collections of manual records become difficult to search and manage.

Problem Definition: To design and implement a Core Java command-line application that enables users to record, manage, persist, analyse, and monitor personal expenses while providing budget tracking and spending alerts through a simple and efficient interface.

3. FUNCTIONAL REQUIREMENTS

3.1 Expense Entry Manager

- Add Expense: accepts amount, category, description, and date; validates the information; and creates an Expense object.
- View Expenses: retrieves stored records and displays them in readable form.
- Update Expense: locates a transaction using its identifier and modifies its details.
- Delete Expense: removes a selected transaction from the active collection and persistent storage.
- Persistent Storage: writes changes to a CSV file so records survive application restarts.

3.2 Budget Setter & Alerts

- Set Budget: defines a spending limit.
- Calculate Spending: totals the amounts in stored expense records.
- Compare with Budget: determines whether expenditure is within, approaching, or above the configured limit.
- Budget Warning: provides a warning when spending reaches a high percentage of the budget.
- Budget Exceeded Alert: reports the amount by which expenditure exceeds the budget.

3.3 Analytics & Summary Generator

- Total Expense: calculates overall expenditure.
- Category-Wise Analysis: groups expenditure by predefined categories.
- Budget Summary: presents budget, spending, remaining amount, and utilization percentage.
- Readable Reporting: formats calculations as a console summary.

4. NON-FUNCTIONAL REQUIREMENTS

4.1 Performance

The application should process normal personal expense datasets quickly. Because the intended use is individual expense management, the expected dataset size is modest, making in-memory collections and CSV processing appropriate for the project's scope.

4.2 Persistence

Expense records must remain available after program termination. CSV File I/O is used to save and reload data.

```
id,date,category,description,amount 1,2026-09-01,FOOD,Lunch,180.00
2,2026-09-02,TRANSPORT,Bus Fare,50.00
```

4.3 Usability

The CLI provides clear numbered menus, understandable prompts, validation messages, confirmation messages, and readable summaries.

4.4 Resource Efficiency

The system uses standard Java facilities and does not require a database server, network connection, or heavyweight framework. This keeps installation, storage, memory use, and deployment requirements low.

5. SYSTEM ARCHITECTURE

The project follows a three-tier architecture. The separation ensures that user interaction, business rules, and persistent storage remain logically independent.

5.1 Presentation Layer

Main.java manages console interaction, menus, user input, navigation, and display of results. It delegates processing to the service layer.

5.2 Business Logic Layer

ExpenseService.java performs CRUD operations, totals, category analysis, budget checks, and summary generation. Budget.java represents budget rules, while ValidationUtils.java centralizes validation.

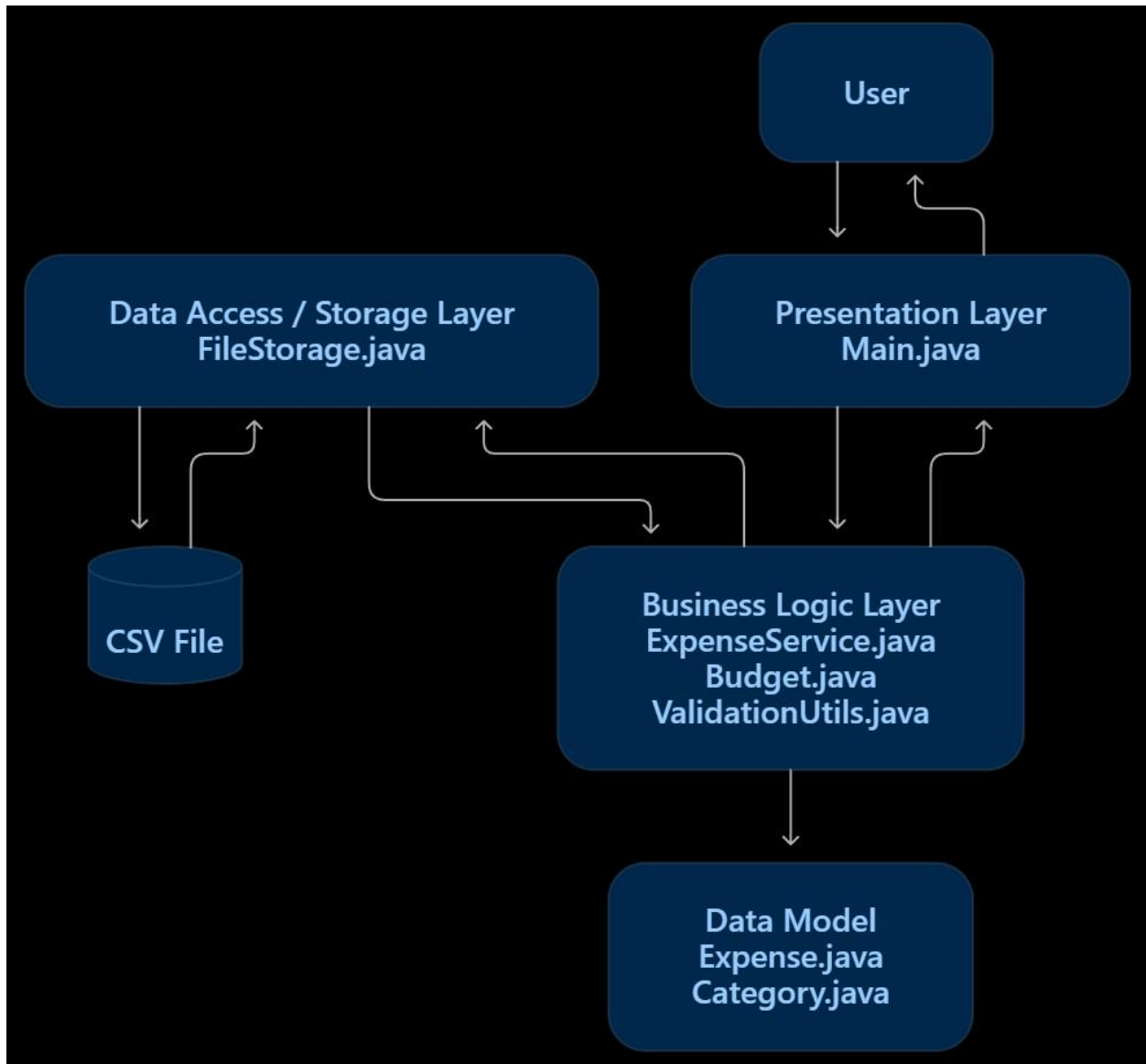
5.3 Data Access / Storage Layer

FileStorage.java reads and writes CSV data and converts between stored records and Java objects.

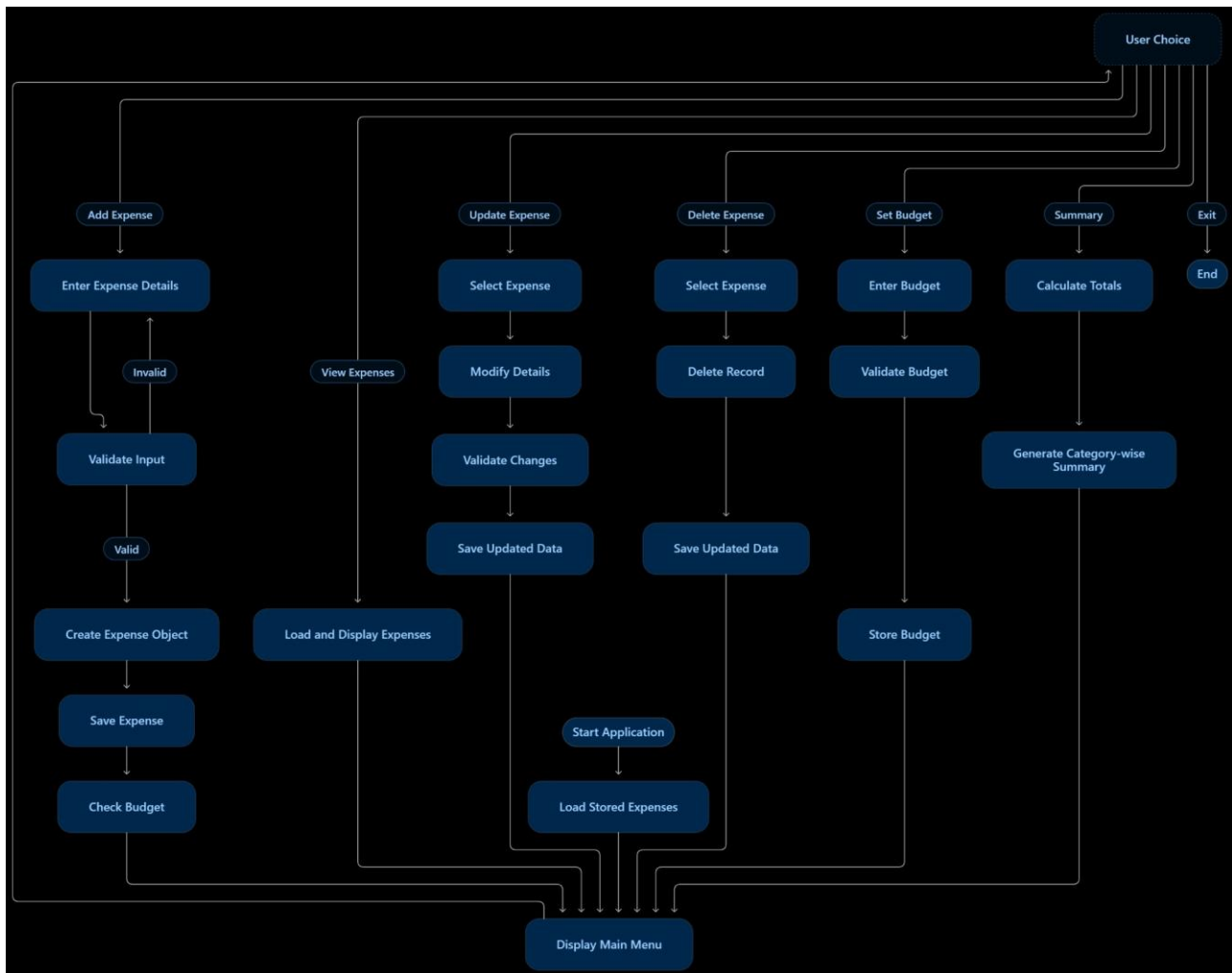
Architecture advantages: separation of responsibilities, easier debugging, maintainability, testability, and a clear path to future GUI or database integration.

6. DESIGN DIAGRAMS

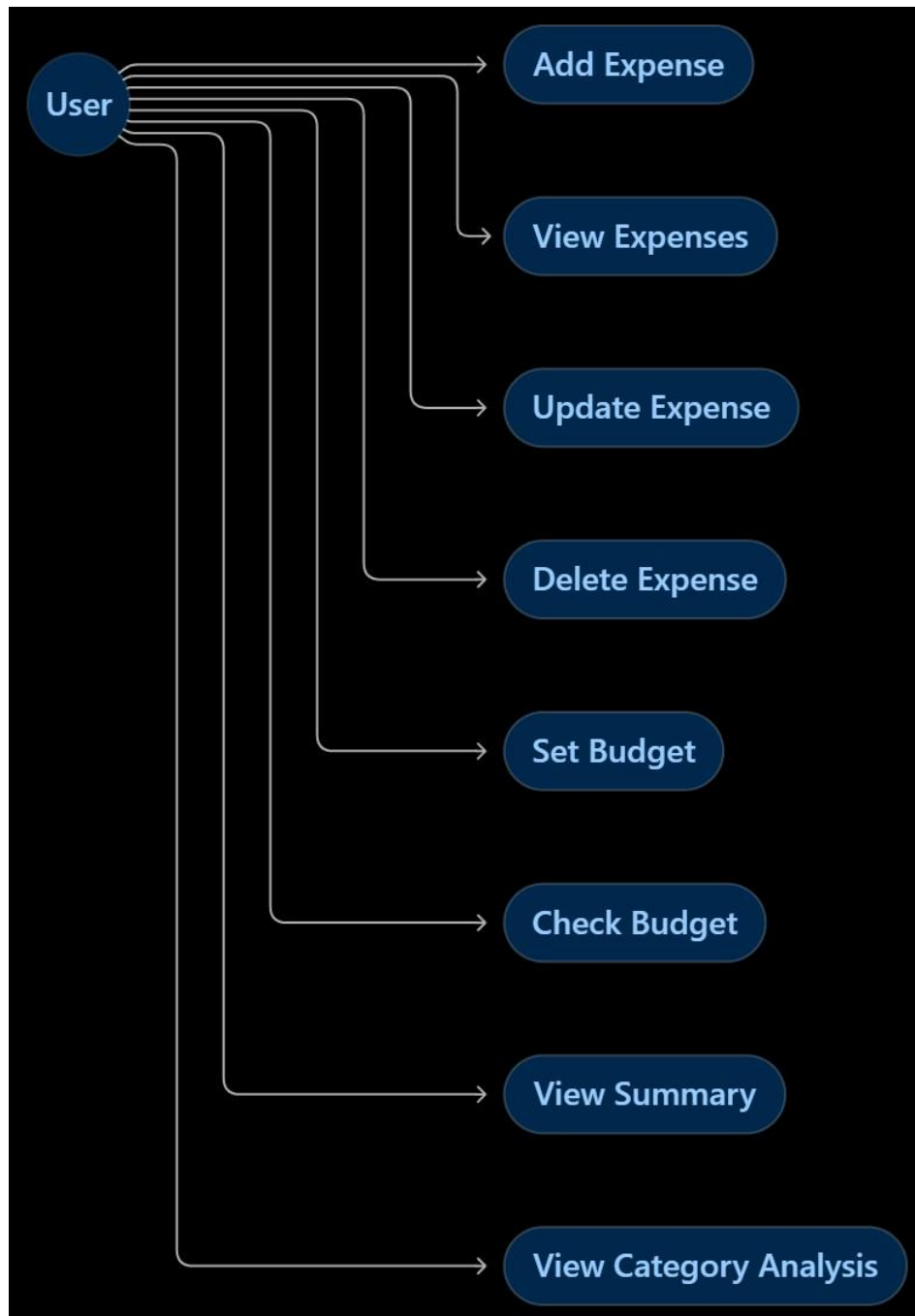
6.1 System Architecture Diagram



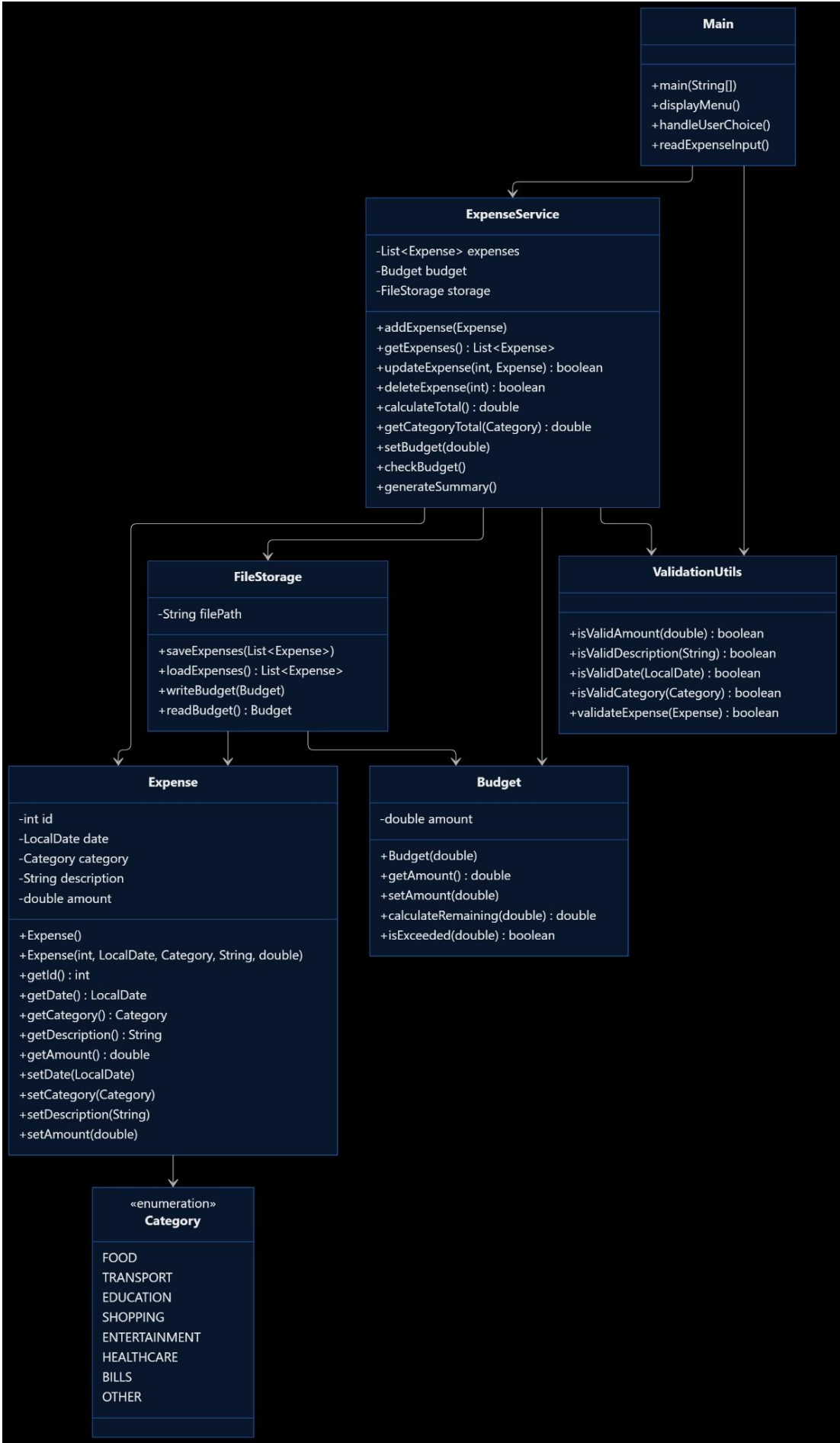
6.2 Process Flow / Workflow Diagram



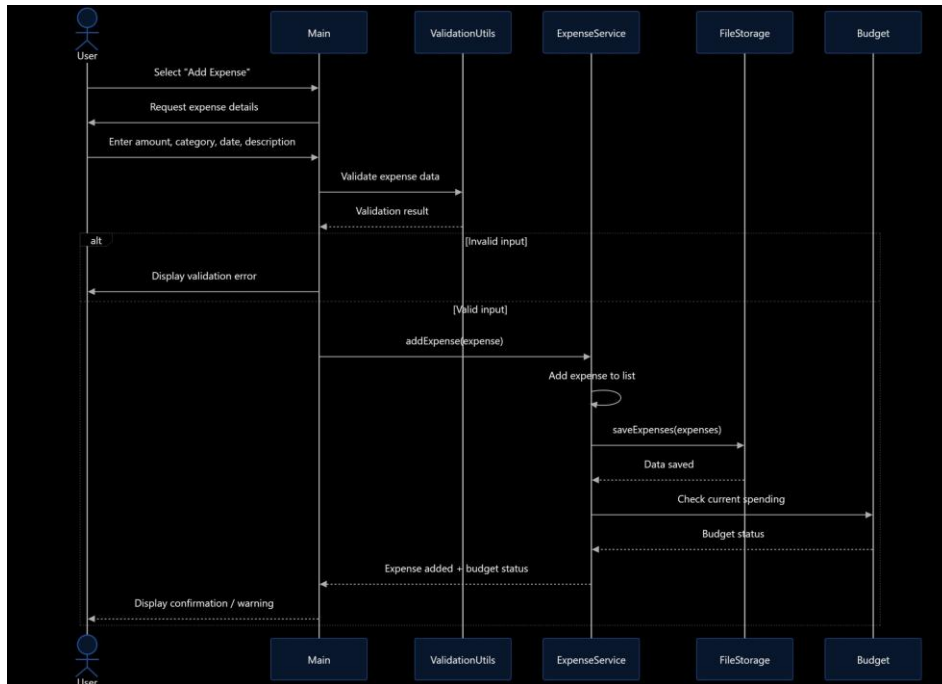
6.3 Use Case Diagram



6.4 Class Diagram — All Seven Java Classes



6.5 Sequence Diagram — Adding Expense & Budget Check



7. DESIGN DECISIONS & RATIONALE

7.1 File I/O Using CSV Instead of SQL

CSV was selected because the application is lightweight and focused on Core Java. CSV files can be manipulated with standard Java I/O classes without a database server or external database configuration.

- Simplicity: straightforward file creation, reading, and writing.
- No database installation: suitable for a standalone academic project.
- Portability: records can be inspected with common text or spreadsheet tools.
- Suitable scale: adequate for a small personal transaction dataset.
- Core Java focus: emphasizes File I/O and Java programming concepts.

CSV is less suitable for very large datasets, complex queries, concurrent access, and multi-user systems. The storage layer can therefore be replaced by a database in a future version.

7.2 Enums for Expense Categories

The Category enum provides a controlled set of valid categories. Using Category.FOOD instead of unrestricted strings improves type safety, prevents inconsistent spellings, and simplifies category-wise aggregation.

7.3 Modular Class Structure

The seven-class structure separates responsibilities: Expense represents data, Category defines categories, Budget handles budget concepts, Expense Service implements business logic, File Storage handles persistence, Validation Utils handles validation, and Main manages interaction.

8. IMPLEMENTATION DETAILS

8.1 Expense.java

Expense.java represents one financial transaction. Its attributes include id, date, category, description, and amount. Private fields with public accessors demonstrate encapsulation.

```
public class Expense {  
    private int id; private  
    LocalDate date; private  
    Category category; private  
    String description; private  
    double amount; }
```

8.2 Category.java

Category.java defines the permitted expense categories.

```
public enum Category { FOOD,  
    TRANSPORT, EDUCATION, SHOPPING,  
    ENTERTAINMENT, HEALTHCARE, BILLS, OTHER  
}
```

8.3 Budget.java

Budget.java stores the configured spending limit and provides calculations for remaining budget and budget-exceeded status.

```
public double calculateRemaining(double spent) {  
    return amount - spent; }
```

8.4 ExpenseService.java

ExpenseService.java is the central business-logic component. It coordinates the expense collection, budget object, storage operations, validation, CRUD operations, total calculations, category totals, budget checks, and summary generation.

8.5 FileStorage.java

FileStorage.java is responsible for persistent storage. It reads CSV records, parses fields, constructs Java objects, and writes updated objects back to the file.

8.6 ValidationUtils.java

ValidationUtils.java centralizes validation for amounts, descriptions, dates, categories, and complete expense objects. It prevents invalid data from reaching the business and storage layers.

8.7 Main.java

Main.java is the program entry point. It displays the menu, reads input, invokes Expense Service operations, presents results, handles invalid choices, and maintains the application loop.

9. SCREENSHOTS / RESULTS

The following sample console interactions represent the expected results of the application and can be used as a reference when inserting actual screenshots.

9.1 Main Menu

```
=====
PERSONAL EXPENSE & BUDGET TRACKER
=====
1. Add Expense
2. View Expenses
3. Update Expense
4. Delete Expense
5. Set Budget
6. View Budget
7. Generate Summary
8. Exit

Enter your choice: 1
```

9.2 Adding an Expense

```
----- ADD EXPENSE -----
Enter expense amount: 250
Select Category: 1
Enter description: Dinner
Enter date (YYYY-MM-DD): 2026-09-14

Expense added successfully.
Expense ID: 5
Amount: INR 250.00
Category: FOOD
Description: Dinner
Date: 2026-09-14
```

9.3 Viewing Expenses

```
ID DATE CATEGORY DESCRIPTION AMOUNT
-----
1 2026-09-01 FOOD Lunch INR 180.00
2 2026-09-02 TRANSPORT Bus Fare INR 50.00
3 2026-09-05 EDUCATION Books INR 850.00
4 2026-09-10 SHOPPING Stationery INR 300.00
```

5 2026-09-14 FOOD Dinner INR 250.00

Total Expense: INR 1,630.00

9.4 Setting a Budget

Enter monthly budget: 2000
Budget successfully set.
Budget Amount: INR 2,000.00
Current Spending: INR 1,630.00
Remaining Budget: INR 370.00

9.5 Budget Warning

Current Spending: INR 1,750.00
Budget: INR 2,000.00
Remaining: INR 250.00 WARNING: You have used
87.50% of your budget.
Please monitor your spending.

9.6 Budget Exceeded

Current Spending: INR 2,150.00
Budget: INR 2,000.00
ALERT: Budget exceeded!
Amount Over Budget: INR 150.00

9.7 Summary Generation

Total Budget : INR 2,000.00
Total Spent : INR 1,750.00
Remaining Budget : INR 250.00
Budget Utilization : 87.50%

FOOD : INR 430.00
TRANSPORT : INR 50.00
EDUCATION : INR 850.00
SHOPPING : INR 300.00
OTHER : INR 120.00

```

Personal Expense & Budget Tracker
1. Add expense
2. View expenses
3. Edit expense
4. Delete expense
5. Set monthly budget
6. View budget alerts
7. View analytics
8. Exit
Choose an option: 1
Amount: 2000
Categories:
Food
Rent
Utilities
Entertainment
Transportation
Healthcare
Education
Shopping
Other
Category: healthcare
Date (yyyy-mm-dd): 2026-09-14
Description: for family

=== Personal Expense & Budget Tracker ===
1. Add expense
2. View expenses
3. Edit expense
4. Delete expense
5. Set monthly budget
6. View budget alerts
7. View analytics
8. Exit
Choose an option: 6
Month (yyyy-MM): 2026-09
ALERT: Food exceeded by 1500.00

=== Personal Expense & Budget Tracker ===
1. Add expense
2. View expenses
3. Edit expense
4. Delete expense
5. Set monthly budget
6. View budget alerts
7. View analytics
8. Exit
Choose an option: 7
Month (yyyy-MM): 2026-09

Analytics for 2026-09
Total: 9000.00
Food: 6500.00
Healthcare: 2500.00
ALERT: Food exceeded by 1500.00
```

10. TESTING APPROACH

10.1 Unit Testing

Unit testing evaluates individual methods and classes independently. Tests include amount validation, description validation, date validation, budget calculations, category totals, and CRUD methods.

10.2 Integration Testing

Integration testing verifies that components work correctly together, including Expense with Expense Service, Expense Service with File Storage, CSV loading into Expense objects, and Expense Service with Budget.

10.3 Boundary Testing

Boundary testing checks extreme and special inputs.

Test Case	Input	Expected Result
Zero amount	0	Reject
Negative amount	-100	Reject
Small positive amount	0.01	Accept
Empty description	Empty	Reject
Budget exactly reached	Spent = Budget	Correct status
Budget exceeded	Spent > Budget	Alert
Empty CSV	No records	Handle empty state
Invalid date	2026-99-99	Reject
Invalid menu choice	99	Display error

11. CHALLENGES FACED

11.1 CSV Parsing and Comma Escaping

CSV parsing becomes challenging when a description contains a comma. A naive split operation can interpret the comma inside the description as a field separator.

```
1,2026-09-14,FOOD,"Lunch, dinner",250
```

The solution is to use proper CSV quoting and escaping logic rather than blindly splitting every line on commas.

11.2 List/File Synchronization

The in-memory List and the persistent CSV file must remain consistent. A controlled save-after-modification workflow ensures that updates and deletions are reflected in persistent storage.

```
Load File → Modify List → Validate → Write Updated List → Confirm
```

11.3 Input Validation

The CLI must handle text entered where numeric input is expected, negative amounts, empty descriptions, invalid dates, invalid categories, and unsupported menu choices. Centralized validation in `ValidationUtils.java` keeps this logic consistent.

12. LEARNINGS & KEY TAKEAWAYS

- Object-Oriented Programming: practical use of classes, objects, encapsulation, constructors, methods, and relationships.
- Java Collections: use of List and iteration to manage multiple expense objects.
- File I/O: reading, writing, parsing, persistence, and file-related exception handling.
- Error Handling: validation and exception handling for invalid input and file failures.
- Enums: representing fixed expense categories in a type-safe way.
- Modular Design: separating data, storage, validation, business logic, and presentation.
- Real-World Logic: translating budget monitoring into programmatic calculations and alerts.
- Testing: considering normal, integration, and boundary cases.

13. FUTURE ENHANCEMENTS

13.1 JavaFX Graphical User Interface

The CLI can be extended with a JavaFX interface containing dashboards, forms, expense tables, date selectors, budget indicators, and interactive summaries.

13.2 Database Integration

CSV storage can be replaced by SQLite, MySQL, or PostgreSQL for larger datasets, structured queries, and more advanced persistence.

13.3 Visual Chart Generation

The analytics module could generate pie charts for category distribution, bar charts for monthly expenditure, line charts for spending trends, and budget-utilization charts.

13.4 Monthly and Yearly Reports

Future versions can generate daily, weekly, monthly, and yearly reports.

13.5 Advanced Budget Alerts

Customizable thresholds could provide information and warnings as spending approaches the configured limit.

13.6 Exportable Reports

The application could export summaries to PDF, Excel, or other structured formats.

13.7 Search and Filtering

Users could filter transactions by date range, category, amount, or description.

14. **REFERENCES**

Oracle. Java Documentation — Java Platform, Standard Edition.

Oracle. Java Language Specification.

Oracle. Java Platform API Documentation.

Oracle. Java I/O and NIO Documentation.

Oracle. Java Collections Framework Documentation.

Oracle. Java Exception Handling Documentation.

Programming in Java — University course guidelines and laboratory/project instructions.

University-provided Programming in Java course material and lecture notes.

Java programming laboratory exercises and academic reference material provided as part of the course.

15. **PROJECT CONCLUSION**

The Personal Expense & Budget Tracker demonstrates how Core Java can be used to build a practical command-line application for personal financial management. It addresses common limitations of manual expense tracking through structured CRUD operations, persistent CSV storage, budget monitoring, alerts, and summary generation.

The project demonstrates practical knowledge of Java OOP, collections, enums, File I/O, exception handling, input validation, modular programming, and software testing. Its three-tier organization separates presentation, business logic, and data storage responsibilities, improving maintainability and providing a clear path toward future GUI and database versions.

The final implementation serves both as a functional expense-management utility and as an academic demonstration of applying Java programming concepts to a real-world software problem.